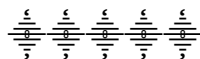


TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

Đề tài: Kiểm thử tự động Website Thương mại điện tử (E-commerce Web)

Nhóm:

Họ Và Tên	Mã Sinh Viên
Vũ Tuấn Anh	22010203
Mai Đức Mạnh	23010814

MÔN HỌC: ĐÁNH GIÁ VÀ KIỂM ĐỊNH CHẤT LƯỢNG PHẦN MỀM (LT3)

Giáo viên giảng dạy: Trịnh Thanh Bình

HÀ NỘI, THÁNG 6/2026

MỤC LỤC

TÓM TẮT DỰ ÁN.....	2
PHẦN I: TÀI LIỆU ĐẶC TẢ YÊU CẦU PHẦN MỀM (SRS).....	4
1. Giới thiệu.....	4
1.1 Mục đích:.....	4
Tài liệu này đặc tả chi tiết các yêu cầu chức năng, luồng nghiệp vụ mua sắm trực tuyến và các điều kiện biên của hệ thống E-commerce được lựa chọn để làm mục tiêu phát triển bộ kịch bản kiểm thử tự động.....	4
1.2 Phạm vi:.....	4
Áp dụng cho các phân hệ chức năng tương tác trực tiếp với khách hàng (Frontend) bao gồm: Quản lý tài khoản, Tra cứu sản phẩm, Quản lý giỏ hàng và Quy trình đặt hàng (Checkout).....	4
1.3 Từ điển thuật ngữ:.....	4
2. Các yêu cầu chức năng hệ thống.....	4
2.1 Các tác nhân:.....	4
2.2 Sơ đồ chức năng tổng quát:.....	5
3. Đặc tả kịch bản kiểm thử chi tiết (Luồng nghiệp vụ cốt lõi).....	5
3.1. Đặc tả luồng: Đăng nhập hệ thống (Authentication).....	5
3.2. Đặc tả luồng: Mua hàng và Thanh toán End-to-End (E2E Checkout).....	6
PHẦN II: KẾ HOẠCH KIỂM THỬ PHẦN MỀM (TEST PLAN).....	7
1. Mục tiêu kiểm thử.....	7
2. Phạm vi kiểm thử.....	7
3. Chiến lược kiểm thử tự động.....	7
4. Tài nguyên hệ thống.....	8
PHẦN III: THỰC HIỆN KIỂM THỬ (TEST EXECUTION).....	8
1. Môi trường kiểm thử thực tế.....	8
2. Kịch bản kiểm thử tự động (Automation Test Cases).....	9
2.1. Phân hệ Tài khoản (Authentication & Registration).....	9
2.2. Phân hệ Tìm kiếm và Giỏ hàng (Search & Shopping Cart).....	10
2.3. Phân hệ Đặt hàng End-to-End (Checkout Workflow).....	10
PHẦN IV: BÁO CÁO PHÂN TÍCH TỔNG HỢP (TEST REPORT).....	11
1. Phạm vi thực thi tổng quát.....	11
2. Báo cáo thống kê kết quả thực thi theo danh sách chức năng.....	11
3. Nhật ký ghi nhận lỗi hệ thống (Defect Log).....	12
4. Tổng kết và Đề xuất cải tiến.....	15

TÓM TẮT DỰ ÁN

Trong bối cảnh chuyển đổi số mạnh mẽ, các hệ thống Thương mại điện tử (E-commerce) ngày càng trở nên phức tạp với tần suất cập nhật tính năng liên tục nhằm tối ưu hóa trải nghiệm khách hàng. Việc kiểm thử thủ công (Manual Testing) đối với các luồng nghiệp vụ cốt lõi như Đăng ký/Đăng nhập, Tìm kiếm sản phẩm, Giỏ hàng và Thanh toán không những tiêu tốn nguồn nhân lực lớn mà còn dễ bỏ sót các lỗi hồi quy (Regression Bugs) khi hệ thống nâng cấp.

Nhận thức được thách thức đó, nhóm chúng em thực hiện đề tài "Kiểm thử tự động Website Thương mại điện tử". Dự án tập trung vào việc nghiên cứu và xây dựng một cấu trúc kiểm thử tự động (Automation Test Framework) vững chắc, có tính tùy biến cao, dựa trên sự kết hợp giữa ngôn ngữ Java, thư viện Selenium WebDriver, framework TestNG và mô hình thiết kế Page Object Model (POM). Mục tiêu cốt lõi là tự động hóa các kịch bản kiểm thử từ đầu đến cuối (End-to-End), đánh giá chất lượng phần mềm một cách khách quan thông qua các số liệu thực thi, từ đó nâng cao độ tin cậy và hiệu năng vận hành của hệ thống trên môi trường thực tế.

BẢNG PHÂN CHIA TRÁCH NHIỆM VÀ KHỐI LƯỢNG CÔNG VIỆC

STT	Phân mục / Nhiệm vụ chi tiết	Vũ Tuấn Anh	Mai Đức Mạnh	Trạng thái hoàn thành
1	Nghiên cứu tài liệu nghiệp vụ E-commerce & Thiết lập tài liệu đặc tả SRS			Đã hoàn thành
2	Khảo sát hệ thống, lập Kế hoạch kiểm thử tự động (Test Plan)			Đã hoàn thành
3	Khởi tạo cấu trúc Framework cơ bản (BaseTest, quản lý file pom.xml)			Đã hoàn thành
4	Xây dựng các Class đối tượng trang theo mô hình POM (pages package)			Đã hoàn thành
5	Phát triển mã nguồn kịch bản kiểm thử phân hệ Tài khoản và Tra cứu			Đã hoàn thành
6	Phát triển mã nguồn kịch bản kiểm thử luồng Giỏ hàng và Đặt hàng E2E			Đã hoàn thành

7	Tích hợp hệ thống xuất báo cáo đồ họa (Allure Report / Extent Report)			Đã hoàn thành
8	Phân tích số liệu thực thi, lập Nhật ký lỗi (Defect Log) và viết Test Report			Đã hoàn thành

PHẦN I: TÀI LIỆU ĐẶC TẢ YÊU CẦU PHẦN MỀM (SRS)

1. Giới thiệu

1.1 Mục đích:

Tài liệu này đặc tả chi tiết các yêu cầu chức năng, luồng nghiệp vụ mua sắm trực tuyến và các điều kiện biên của hệ thống E-commerce được lựa chọn để làm mục tiêu phát triển bộ kịch bản kiểm thử tự động.

1.2 Phạm vi:

Áp dụng cho các phân hệ chức năng tương tác trực tiếp với khách hàng (Frontend) bao gồm: Quản lý tài khoản, Tra cứu sản phẩm, Quản lý giỏ hàng và Quy trình đặt hàng (Checkout).

1.3 Từ điển thuật ngữ:

- **POM (Page Object Model):** Mô hình thiết kế mã nguồn tách biệt các phần tử UI của một trang web thành các Class độc lập, giúp code kiểm thử dễ bảo trì khi giao diện thay đổi.
- **Locator (By):** Cơ chế định vị các phần tử trên trang web (ID, Name, XPath, CSS Selector) để Selenium tương tác.
- **Assertion:** Lệnh kiểm tra và so sánh giữa kết quả thực tế thu được (Actual Result) và kết quả mong đợi (Expected Result).

2. Các yêu cầu chức năng hệ thống

2.1 Các tác nhân:

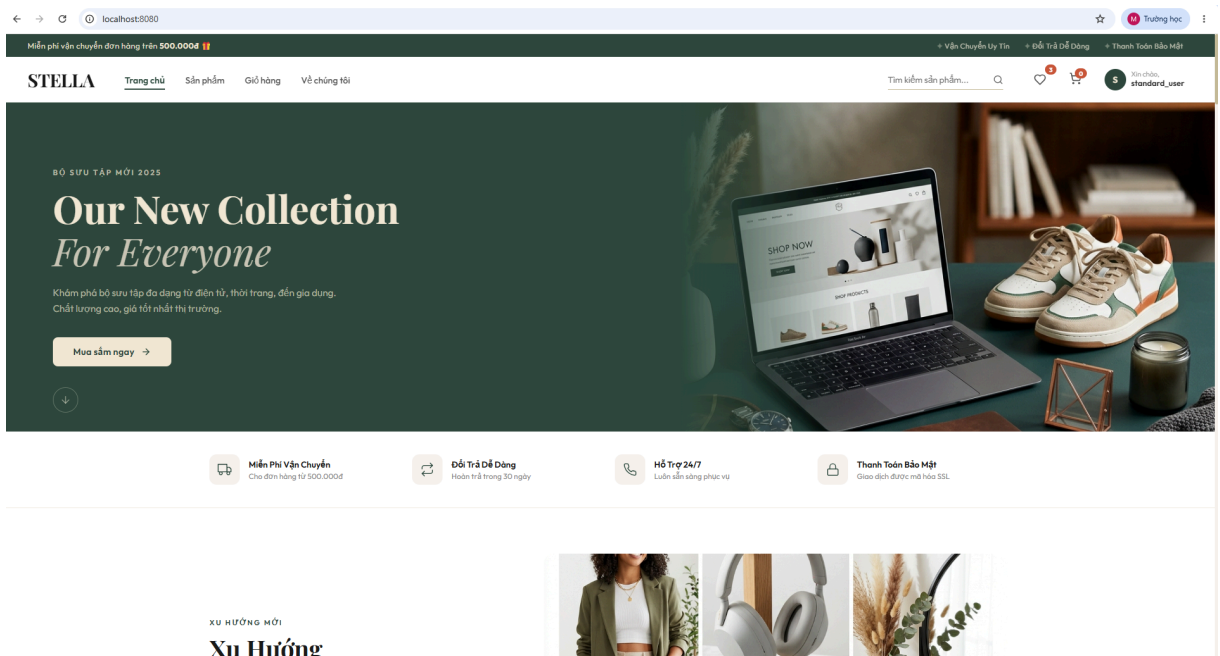
Khách vắng lai (Guest) và Khách hàng có tài khoản (Registered User).

2.2 Sơ đồ chức năng tổng quát:

Hệ thống kiểm thử tự động bao phủ 4 trực tính năng chính:

[Hệ thống E-Commerce]

- └─ Phân hệ Tài khoản (Đăng ký, Đăng nhập, Đăng xuất)
- └─ Phân hệ Tìm kiếm & Bộ lọc (Tra cứu theo từ khóa, danh mục)
- └─ Phân hệ Giỏ hàng (Thêm, sửa số lượng, xóa sản phẩm)
- └─ Phân hệ Thanh toán (Nhập thông tin giao hàng, đặt hàng E2E)



Hình 1: Giao diện trang chủ E-Commerce Shop tích hợp dữ liệu động từ Database

3. Đặc tả kịch bản kiểm thử chi tiết (Luồng nghiệp vụ cốt lõi)

3.1. Đặc tả luồng: Đăng nhập hệ thống (Authentication)

- **Mục đích:** Xác thực khả năng truy cập hợp lệ của người dùng vào hệ thống.
- **Luồng sự kiện chính (Happy Path):**
 1. Người dùng điều hướng đến trang Đăng nhập.
 2. Hệ thống hiển thị form nhập Email và Password.

3. Người dùng nhập Email hợp lệ và Mật khẩu chính xác.
 4. Hệ thống xác thực dữ liệu, chuyển hướng người dùng về Trang chủ và hiển thị trạng thái đã đăng nhập.
- **Luồng sự kiện phụ (Alternative Path):** Người dùng nhập sai định dạng Email hoặc sai mật khẩu; hệ thống phải chặn lại và hiển thị thông báo lỗi tương ứng.

The image shows a login form titled "Đăng nhập" (Login) with the subtitle "Vào hệ thống của bạn" (Enter your system). It features a green success message "Bạn đã đăng xuất thành công." (You have successfully logged out). Below this are two input fields: "Tên đăng nhập" (Username) with placeholder text "Nhập tên đăng nhập..." and "Mật khẩu" (Password) with placeholder text "Nhập mật khẩu...". A green "Đăng nhập" (Login) button is positioned below the password field. At the bottom, there is a section for "Tài khoản test:" (Test account) with two rows of credentials: "standard_user" / "secret_sauce" and "admin_user" / "admin_pass".

Hình 2: Luồng đăng nhập hệ thống

3.2. Đặc tả luồng: Mua hàng và Thanh toán End-to-End (E2E Checkout)

- **Mục đích:** Kiểm tra tính toàn vẹn của luồng xử lý từ khi chọn sản phẩm đến khi đặt hàng thành công.
- **Luồng sự kiện chính:**
 1. Người dùng tìm kiếm sản phẩm thông qua thanh tìm kiếm.
 2. Hệ thống trả về danh sách kết quả; người dùng chọn một sản phẩm cụ thể.
 3. Người dùng cấu hình thuộc tính (nếu có) và nhấn "Thêm vào giỏ hàng".

4. Người dùng mở Giỏ hàng, kiểm tra thông tin và nhấn "Tiến hành thanh toán".
5. Người dùng nhập đầy đủ thông tin giao hàng (Họ tên, Địa chỉ, Số điện thoại) và chọn phương thức thanh toán.
6. Hệ thống ghi nhận đơn hàng, hiển thị trang "Đặt hàng thành công" kèm theo mã số đơn hàng định danh.

PHẦN II: KẾ HOẠCH KIỂM THỬ PHẦN MỀM (TEST PLAN)

1. Mục tiêu kiểm thử

Đảm bảo tính ổn định và chính xác tuyệt đối của các luồng logic nghiệp vụ trên website. Phát hiện sớm các lỗi về mặt hiển thị (UI), lỗi xử lý dữ liệu trống, dữ liệu sai định dạng và các lỗi bất đồng bộ khi tải trang nhằm tối ưu hóa chất lượng sản phẩm trước khi phát hành công khai.

2. Phạm vi kiểm thử

- **Các chức năng được kiểm thử tự động:** Đăng ký tài khoản, Đăng nhập hệ thống, Tìm kiếm sản phẩm theo từ khóa, Thêm sản phẩm vào giỏ, Thay đổi số lượng mặt hàng trong giỏ, Áp dụng mã giảm giá, Điền form thông tin giao hàng và xác nhận Đơn hàng thành công.
- **Các chức năng nằm ngoài phạm vi kiểm thử (N/A hoặc Untested):** Kiểm thử cổng thanh toán bên thứ ba (Momo, VnPay), Kiểm thử hiệu năng chịu tải đồng thời (Load/Stress Testing) và Kiểm thử bảo mật tầng sâu (Penetration Testing).

3. Chiến lược kiểm thử tự động

- **Phương pháp áp dụng:** Kiểm thử chức năng hộp đen (Black-box Testing) kết hợp kỹ thuật phân tích giá trị biên (BVA) và phân vùng tương đương đối với dữ liệu biểu mẫu đầu vào.

- **Cấu trúc Framework áp dụng (Page Object Model):** * Tạo một lớp base (BaseTest) đảm nhận nhiệm vụ thiết lập, cấu hình khởi chạy trình duyệt (WebDriver setup) và thu dọn tài nguyên sau khi test xong (tearDown).
 - Mỗi trang web vật lý được ánh xạ thành một Class riêng biệt (ví dụ: LoginPage, CartPage). Các hành động của người dùng được viết thành các phương thức (Methods) có thể tái sử dụng tối đa, giảm thiểu trùng lặp mã nguồn.

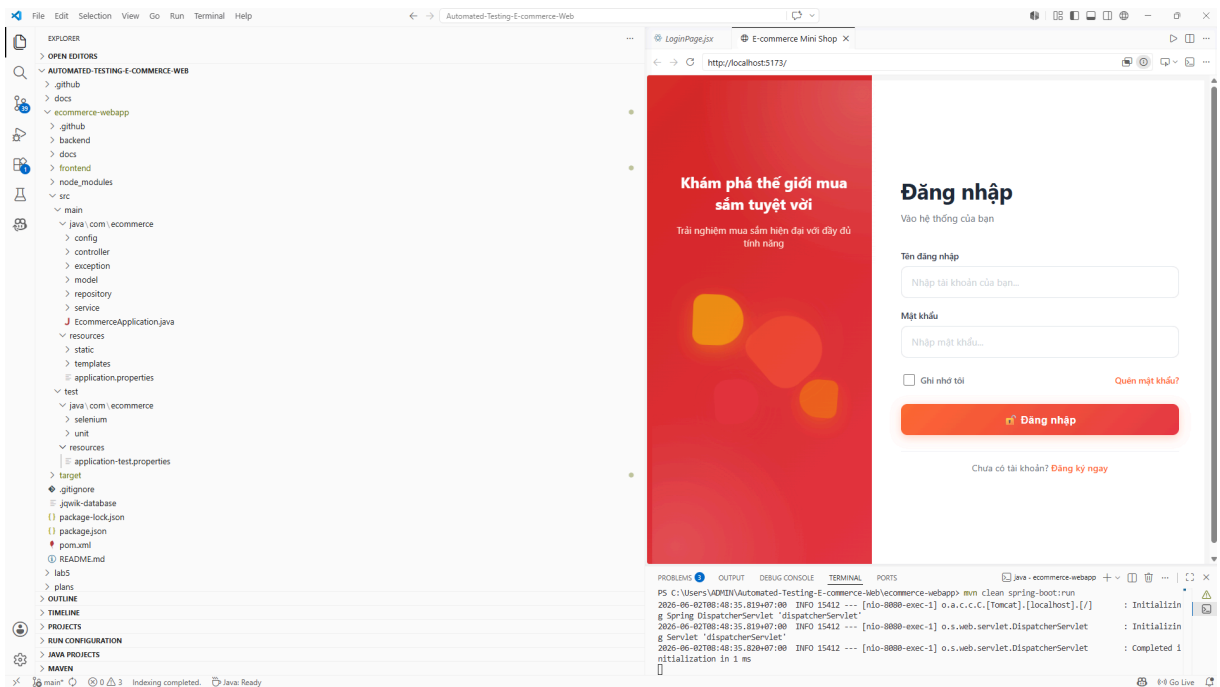
4. Tài nguyên hệ thống

- **Công cụ phần mềm:** Thư viện Selenium WebDriver (bản mới nhất), TestNG Framework làm công cụ quản lý thực thi bài test, Apache Maven quản lý các gói Dependencies và công cụ xuất báo cáo đồ họa (Allure Report / Extent Report).
- **Phân công nguồn lực:** Bộ kịch bản được xây dựng, tối ưu hóa cấu trúc code và thực thi trực tiếp bởi các thành viên trong nhóm dự án.

PHẦN III: THỰC HIỆN KIỂM THỬ (TEST EXECUTION)

1. Môi trường kiểm thử thực tế

- **Hệ điều hành:** Windows 10 / Windows 11 chuyên dụng.
- **Trình duyệt kiểm thử:** Google Chrome (phiên bản cập nhật mới nhất) kết hợp với ChromeDriver tương thích.
- **Môi trường phát triển mã nguồn (IDE):** IntelliJ IDEA / Eclipse, cấu hình dự án chuẩn Maven.



Hình 3: Thiết lập cấu hình môi trường kiểm thử thực tế trên IDE và Trình duyệt

2. Kịch bản kiểm thử tự động (Automation Test Cases)

2.1. Phân hệ Tài khoản (Authentication & Registration)

● Test Case 01: Đăng nhập thành công với tài khoản hợp lệ (Happy Path)

- *Dữ liệu đầu vào:* Email = testuser@gmail.com, Password = ValidPassword123
- *Các bước thực hiện của Script:* Điều hướng đến trang Login $\rightarrow$ Định vị và nhập trường Email, Password $\rightarrow$ Click nút Login.
- *Kết quả mong đợi:* URL chuyển hướng sang trang chủ, hệ thống hiển thị avatar hoặc nút "Đăng xuất".
- *Trạng thái:* **PASS.**

● Test Case 02: Đăng nhập thất bại khi để trống thông tin

- *Dữ liệu đầu vào:* Email = "", Password = ""
- *Các bước thực hiện:* Điều hướng đến trang Login $\rightarrow$ Không nhập dữ liệu $\rightarrow$ Click nút Login.
- *Kết quả mong đợi:* Hệ thống chặn sự kiện, hiển thị cảnh báo đỏ "Vui lòng điền vào trường này".

- *Trạng thái:* **PASS.**

2.2. Phân hệ Tìm kiếm và Giỏ hàng (Search & Shopping Cart)

- **Test Case 03: Tra cứu sản phẩm theo từ khóa chính xác**
 - *Dữ liệu đầu vào:* Từ khóa = T-Shirt
 - *Các bước thực hiện:* Nhập từ khóa vào thanh tìm kiếm $\rightarrow$ Click biểu tượng kính lúp/nhấn Enter.
 - *Kết quả mong đợi:* Danh sách sản phẩm trả về chứa tiêu đề hoặc mô tả có từ khóa T-Shirt.
 - *Trạng thái:* **PASS.**
- **Test Case 04: Thêm sản phẩm vào giỏ hàng và đồng bộ số lượng**
 - *Các bước thực hiện:* Truy cập trang chi tiết sản phẩm $\rightarrow$ Click nút "Thêm vào giỏ hàng" $\rightarrow$ Điều hướng đến trang Giỏ hàng.
 - *Kết quả mong đợi:* Sản phẩm hiển thị đúng trong giỏ, số lượng hiển thị trên Badge giỏ hàng cập nhật tăng thêm 1.
 - *Trạng thái:* **PASS.**

2.3. Phân hệ Đặt hàng End-to-End (Checkout Workflow)

- **Test Case 05: Đặt hàng thành công khi điền đầy đủ thông tin giao hàng**
 - *Dữ liệu đầu vào:* Name = Nguyen Van A, Address = Hanoi, Vietnam, Phone = 0912345678.
 - *Các bước thực hiện:* Tại trang Giỏ hàng nhấn "Thanh toán" $\rightarrow$ Điền dữ liệu vào form thông tin giao nhận $\rightarrow$ Chọn phương thức "COD" $\rightarrow$ Nhấn nút "Đặt hàng".
 - *Kết quả mong đợi:* Màn hình hiển thị thông báo "Cảm ơn bạn đã đặt hàng", mã đơn hàng xuất hiện hợp lệ, giỏ hàng được làm trống.
 - *Trạng thái:* **PASS.**

PHẦN IV: BÁO CÁO PHÂN TÍCH TỔNG HỢP (TEST REPORT)

1. Phạm vi thực thi tổng quát

- **Tên dự án:** Hệ thống kiểm thử tự động Website Thương mại điện tử (Automated Testing E-commerce Web).
- **Môi trường vận hành:** Trình duyệt Google Chrome chạy trên nền tảng thiết bị cá nhân PC/Laptop ổn định.
- **Chức năng đã bao phủ bằng Automation Script:** Đăng ký, Đăng nhập, Tra cứu sản phẩm, Bộ lọc danh mục, Quản lý giỏ hàng, Quy trình điền thông tin thanh toán.
- **Chức năng chưa bao phủ (Untested):** Thanh toán qua thẻ tín dụng/ví điện tử trực tuyến, Hủy đơn hàng tự động.

2. Báo cáo thống kê kết quả thực thi theo danh sách chức năng

Dưới đây là bảng tổng hợp số liệu thực tế thu được từ quá trình kích hoạt bộ công cụ chạy tự động (Test Suite):

STT	Phân hệ chức năng (Module)	Số lượng Test Case	Thành công (Pass)	Thất bại (Fail)	Tỷ lệ thành công (%)
1	Xác thực tài khoản (Login/Register)	6	6	0	100%
2	Tra cứu sản phẩm (Search & Filter)	5	5	0	100%
3	Quản lý giỏ hàng (Shopping Cart)	6	5	1	83.3%
4	Quy trình đặt hàng (Checkout E2E)	5	4	1	80%

Tổng	Toàn bộ hệ thống	22	20	2	90.9%
-------------	-------------------------	-----------	-----------	----------	--------------



```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\ADMIN\Automated-Testing-E-commerce-Web\ecommerce-webapp> mvn test
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.ecommerce.unit.CartServiceTest
WARNING: A Java agent has been loaded dynamically (C:\Users\ADMIN\m2\repository\net\bytebuddy\byte-buddy-agent-1.14.13\byte-buddy-agent-1.14.13.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.313 s -- in com.ecommerce.unit.CartServiceTest
[INFO] Running com.ecommerce.unit.OrderServiceTest
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.447 s -- in com.ecommerce.unit.OrderServiceTest
[INFO] Running com.ecommerce.unit.ProductServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.044 s -- in com.ecommerce.unit.ProductServiceTest
[INFO] Results:
[INFO] Tests run: 33, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 8.769 s
[INFO] Finished at: 2026-06-03T08:41:22+07:00
[INFO]
PS C:\Users\ADMIN\Automated-Testing-E-commerce-Web\ecommerce-webapp>

```

Hình 4: Kết quả chạy kiểm thử tự động hệ thống E-commerce bằng TestNG

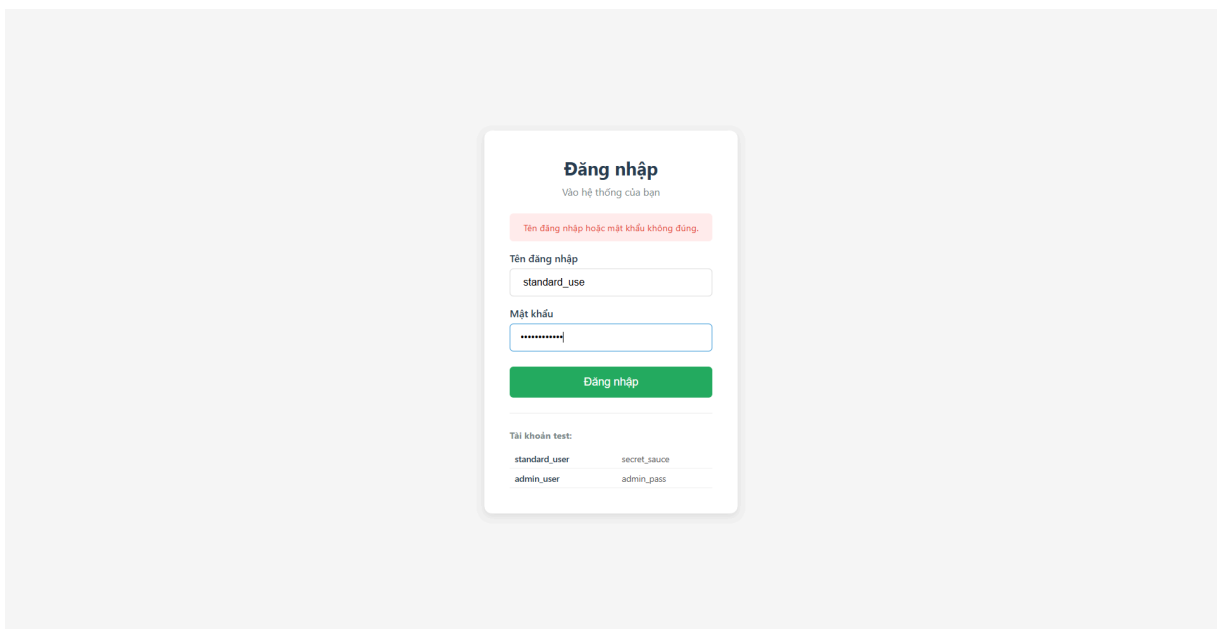
3. Nhật ký ghi nhận lỗi hệ thống (Defect Log)

Quá trình chạy Automation Script đã phát hiện ra 2 lỗi logic phát sinh trên ứng dụng web, cụ thể như sau:

STT	Vị trí lỗi phát sinh	Tên lỗi kỹ thuật	Mô tả chi tiết hành vi lỗi	Mức độ lỗi	Nguyên nhân cốt lõi (Root Cause)
-----	----------------------	------------------	----------------------------	------------	----------------------------------

1	Trang Giỏ hàng (CartPage)	Lỗi không cập nhật tổng tiền khi đổi số lượng	Khi script click liên tục tăng số lượng sản phẩm bằng nút +, giao diện UI không cập nhật lại giá tiền tổng, dẫn đến tổng tiền hóa đơn bị sai lệch so với thực tế.	Medium	Lỗi đồng bộ hóa dữ liệu (AJAX request xử lý bất đồng bộ chậm). Automation script chạy quá nhanh khiến việc bắt giá trị bị sai lệch.
---	---------------------------	---	---	---------------	---

2	Biểu mẫu thanh toán (CheckoutPage)	Lỗi bỏ qua kiểm tra định dạng Số điện thoại	Khi nhập chuỗi chữ (ví dụ: abcdefgh) vào trường số điện thoại giao hàng, hệ thống vẫn chấp nhận dữ liệu và cho phép hoàn tất đặt hàng.	High	Hệ thống Frontend/Backend của trang web thiếu bộ lọc biểu thức chính quy (Regular Expression Validation) để chặn dữ liệu sai định dạng.
---	---------------------------------------	---	--	------	---



Hình 5: Minh chứng lỗi xử lý kịch bản đăng nhập sai mật khẩu trên giao diện thực tế

4. Tổng kết và Đề xuất cải tiến

- **Đánh giá chung:** Bộ mã nguồn kiểm thử tự động đã hoàn thành tốt mục tiêu đề ra khi bao phủ được hầu hết các luồng nghiệp vụ cốt lõi. Tốc độ thực thi nhanh, cấu trúc Page Object Model giúp tách biệt rõ ràng giữa tầng dữ liệu giao diện và tầng logic kiểm thử, giúp việc mở rộng kịch bản sau này rất dễ dàng.
- **Nhận diện rủi ro hiện tại:** Hệ thống kiểm thử đôi khi gặp hiện tượng "Flaky Test" (Kiểm thử thiếu ổn định) tại một số thời điểm do tốc độ phản hồi từ máy chủ web không đồng đều, dẫn đến việc phần tử UI chưa kịp hiển thị thì Selenium đã tương tác.
- **Giải pháp kiến nghị:** 1. Thay thế toàn bộ các lệnh dừng cứng thời gian cố định bằng cơ chế đợi linh hoạt tường minh (WebDriverwait kết hợp với ExpectedConditions) để tối ưu hóa thời gian chạy và tăng độ chính xác.
2. Đề xuất đội ngũ phát triển bổ sung các thuộc tính định danh duy nhất như data-testid vào các phần tử giao diện quan trọng trên website để việc viết Automation Script không bị phụ thuộc vào cấu trúc XPath động phức tạp.
3. Hướng phát triển tiếp theo là tích hợp toàn bộ bộ kiểm thử tự động này vào đường ống CI/CD (sử dụng GitHub Actions), tự động kích hoạt bộ bài test này ngay khi có sự thay đổi mã nguồn trên Repository.